Nikhil Kale
47

Lab - 1: **Experiment No: 1**

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Theory

Cryptography in Blockchain:
Cryptography provides security, integrity, authentication, and immutability in blockchain. Cryptographic hash functions are used to securely represent transaction and block data.

Cryptographic Hash Function:
A hash function converts input data into a fixed-length hash value. In this experiment, **SHA-256** is used. It is deterministic, collision-resistant, and has an avalanche effect, where even a small change in input produces a completely different hash.

Nonce and Cryptographic Puzzle:
A **nonce** is a value repeatedly changed during mining to find a hash satisfying a specific condition, such as starting with a certain number of zeros. Finding the correct nonce is called solving a **cryptographic puzzle** and forms the basis of Proof-of-Work.

Merkle Tree:
A Merkle Tree is a binary tree of cryptographic hashes. Each transaction is first hashed, and pairs of hashes are repeatedly combined and hashed until a single hash remains. This final hash is called the **Merkle Root**.

Benefits:
Merkle Trees provide efficient transaction verification, data integrity, tamper detection, and reduced storage requirements. They are widely used in blockchain systems such as Bitcoin.

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**

- Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Program 1-

Program #1: Python program that uses the hashlib library to create the hash of a given string:

XXXXXXX #error statement; 3 Error

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                          #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                         #error statement
```

```
Enter a string: Nikhil
Hash: 6b9ec3916da443f959d1cd6392ac1242e3820c57cd9bf9f9b68a4252c9d824c3
```

Program 2-

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                                    # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                          # Error
```

```
Enter a string: Nikhil
Enter the nonce: 1434955
Hash Code: 000009d3b0b8e499e308f65e0640009d02794ce162a3eba73a743f99dd625563
```

Program 3-

```python
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)                    #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):                          #Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Nikhil"
num_zeros = 5

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)                                #Error
```

```
Hash: 000009d3b0b8e499e308f65e0640009d02794ce162a3eba73a743f99dd625563
Input String: Nikhil
Leading Zeros: 5
Expected Nonce: 1434955
```

Program 4-

Program 4: Generating Merkle Tree for given set of Transactions

```python
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash each initial transaction
    hashed_transactions = [hashlib.sha256(t.encode('utf-8')).hexdigest() for t in transactions]
    print(f"Initial Hashed Transactions: {hashed_transactions}")

    # If only one transaction, its hash is the root
    if len(hashed_transactions) == 1:
        return hashed_transactions[0]

    # Recursive construction of the Merkle Tree
    current_level_hashes = hashed_transactions
    level = 1

    while len(current_level_hashes) > 1:
        print(f"\nBuilding Merkle Tree - Level {level}:")

        # If odd number of hashes, duplicate the last one
        if len(current_level_hashes) % 2 != 0:
            current_level_hashes.append(current_level_hashes[-1])

        next_level_hashes = []
        for i in range(0, len(current_level_hashes), 2):
            combined = current_level_hashes[i] + current_level_hashes[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            next_level_hashes.append(hash_combined)
            print(f"  Combining {current_level_hashes[i]} and {current_level_hashes[i+1]} -> {hash_combined}")

        current_level_hashes = next_level_hashes
        level += 1

    return current_level_hashes[0]
```

```python
    return current_level_hashes[0]

# Updated Example usage
transactions = [
    'Alice \u2192 Bob : $200',
    'Bob \u2192 Dave : $500',
    'Dave \u2192 Eve : $100',
    'Eve \u2192 Alice : $300',
    'Roo \u2192 Bob : $50'
]

merkle_root = build_merkle_tree(transactions)
print(f"\nFinal Merkle Root: {merkle_root}")
```

```
Initial Hashed Transactions: ['d7d1c2448f818062ab9bb89aa9f95564670895f927d730d2d43ba8e5cada8a4d', 'ebc396058242fc4cb711aa1dd2c2ce1c44818b9d35c9d53465bee3c12462072e', '56a4

Building Merkle Tree - Level 1:
  Combining d7d1c2448f818062ab9bb89aa9f95564670895f927d730d2d43ba8e5cada8a4d and ebc396058242fc4cb711aa1dd2c2ce1c44818b9d35c9d53465bee3c12462072e -> 51215ac8c39b7d8d72ce87
  Combining 56a42d8ba343cbcf8133e8a0300564876f2e76dfe8281db52fccbad24fc8d88a and fc3d2297f8b8aa145fa4ded66a1ed06db9d46d661a8d3673e1ceb0dbbaf597a6 -> b5e0a195812483b212784(
  Combining 70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a and 70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a -> 7b5c2d1c9a72494009e89(

Building Merkle Tree - Level 2:
  Combining 51215ac8c39b7d8d72ce873c5cf75a9b7d3edd069c79007a1f0827af3e43d258 and b5e0a195812483b212784056d2bf15f8caeff4fb4bde6e4910d9827d039632f1 -> 8e3aa833a93cf5663d513
  Combining 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6 and 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6 -> 5b0369cfb2de9ed3b41d3

Building Merkle Tree - Level 3:
  Combining 8e3aa833a93cf5663d5138a8d80d8e6a8cf006517a462fb4260806795701f45d and 5b0369cfb2de9ed3b41d380675c240323de05bc9b9062fae8cd5991902828cd2 -> 382f188ce8666903c084f5

Final Merkle Root: 382f188ce8666903c084f5704ade4a7e1a1948242bdfd0de2b529d9fedb6fe2f
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.